Software Architecture and Design Specification

Student Attendance Management System (SAMS)

Project	Student Attendance Management System (SAMS)
Version	1.0
Authors	Atul Kandiyil (PES2UG24CS920), Riona Jeena Dsouza (PES2UG24CS654), Amit (PES2UG24CS624), Shivakumar (PES2UG24CS548) - Team 1
Date	08-10-2026
Status	Draft
Stack	Python / Django, MySQL, REST (JSON over HTTPS)

Revision History

Version	Date	Author	Change Summary
1.0	08-10-2026	Team 1	Initial SAD draft derived from SRS v1.0 (13-09-2026) and STP v1.0 (16-09-2026)

Approvals

Role	Name	Signature / Date
Course Coordinator		
Team Lead	Atul Kandiyil	
Dev Lead	Riona Jeena Dsouza	
QA Lead	Shivakumar	

1. Introduction

1.1 Purpose

This document describes how we plan to build the Student Attendance Management System (SAMS). We took the requirements from our SRS and worked out the architecture and design that the team will follow while coding and that the testers can check against the Software Test Plan (STP). Here you will find the main components, the technology we picked and why, how the parts talk to each other, the key flows drawn as sequence diagrams, and how we keep student data safe.

1.2 Scope

SAMS covers everything the SRS asks for. Users log in with role-based access. Faculty take attendance for each class, either by manual roll call, by QR self check-in, or with optional biometric/RFID devices. The system stores the records, lets faculty fix mistakes within a time limit, works out attendance percentages, flags students who fall below the threshold, produces reports and sends low-attendance alerts. Admins manage courses, users, policies and leave requests. SAMS also connects to the college's Student Information System (SIS) and to an SMS/email gateway.

Not covered: payroll, fees, examinations, hostel and learning-management features, and academic records that are not about attendance, such as grades and transcripts.

1.3 Audience

This document is meant for developers, QA engineers, database administrators, security reviewers, faculty coordinators, college administrators, course instructors and evaluators, and whoever maintains the system later.

1.4 Definitions

Term	Meaning
SAMS	Student Attendance Management System
SIS	Student Information System (institutional system of record for enrolment)
RBAC	Role-Based Access Control
TLS	Transport Layer Security
JWT	JSON Web Token used for stateless API authentication
QR / RFID	Quick Response code / Radio-Frequency Identification
OTP	One-Time Password
ADR	Architecture Decision Record
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
WCAG	Web Content Accessibility Guidelines
RTM	Requirements Traceability Matrix
ORM	Object-Relational Mapper (Django ORM)

2. Document Overview

2.1 How to use this document

Section 3 covers the architecture: goals, stakeholders, the component diagram, the pattern we chose, technology, risks, traceability and security. Section 4 covers the detailed design: data entities, three sequence diagrams for real SAMS flows, the API, error handling and monitoring, the user interface and open issues. Section 5 has the glossary, references and tools. If you are a developer, read Sections 3.3 to 3.6 and 4.2 to 4.3 first. If you are testing, start with the traceability table in 3.8 and the API details in 4.3. Reviewers will mostly care about the security architecture in 3.9.

2.2 Related Documents

Document	Version / Date	Relationship to this SAD
Software Requirements Specification (SRS)	1.0 / 13-09-2026	Source of all functional (SAMS-F), non-functional (SAMS-NF) and security (SAMS-SR) requirements traced in Section 3.8
Software Test Plan (STP)	1.0 / 16-09-2026	Defines test levels, environment (Django, MySQL, JMeter, Postman) and test cases that verify this design
Requirements Traceability Matrix (RTM)	SRS Section 8	Maps requirements to design specs (DS-*), modules and test cases; extended in Section 3.8

3. Architecture

3.1 Goals & Constraints

Architectural goals

Goal	Target / Measure	SRS source
Performance	Attendance submission confirmed within 3 s for 90% of requests at up to 80 students per session	SAMS-NF-001
Reliability / availability	99.5% monthly availability during academic hours (excluding planned maintenance); tolerate brief SIS outages via roster cache	SAMS-NF-002, SRS 3.4
Security & privacy	Encrypt personal and biometric data at rest and in transit; RBAC enforced at API layer; full audit trail	SAMS-NF-003, SAMS-SR-001..005
Auditability & retention	Immutable log of every create/edit/override; records retained at least 5 years	SAMS-NF-004, SAMS-SR-004
Usability & scalability	WCAG 2.1 AA; at least 500 concurrent users without degraded response	SAMS-NF-005
Modularity / maintainability	Independent modules with clear interfaces so features can be changed or tested in isolation	Developers / STP

Constraints

- SAMS has to work with the college's existing SIS for enrolment and class rosters. It must not turn into a second place where student identity is stored.
- We must follow the institution's data-protection and student-privacy rules, especially for biometric data, which is stored only as encrypted templates.
- Faculty can correct attendance only within 48 hours, and every correction is logged. Admins can override later, but they must give a reason.
- The system has to run on the college's normal network and work in the supported browsers (Chrome, Firefox, Edge and Safari). Hardware readers are optional and depend on the vendor.
- We build with Python (Django) and a relational database, since that matches the team's skills and the test setup in the STP.

3.2 Stakeholders & Concerns

Stakeholder	Primary concerns	Addressed by
Students	Simple, fast, mobile-friendly view of own attendance; privacy of personal data; fair handling of leave requests	Responsive UI (4.5), RBAC, leave workflow (AdminModule)
Faculty	Quick marking (<3 s), ability to fix mistakes within the window, class analytics	AttendanceModule, CorrectionModule, ReportModule
Administrators	Policy configuration, institution-wide reports, overrides, user management	AdminModule, ReportModule, audit log
Guardians	Timely low-attendance alerts without needing a login	NotificationModule, Notification Gateway
Institution / Regulators	Data protection, audit and accreditation evidence, 5-year retention	Security architecture (3.9), AuditService
Developers	Modularity, clear API contracts, testability	Layered design, API specs (4.3)
QA Engineers	Traceable requirements, testable interfaces, stable test data	Traceability (3.8), STP alignment
Database Admins / Ops	Backups, monitoring, archival, availability	Data layer, monitoring (4.4)

3.3 Component (UML) Diagram

Figure 1 shows the main parts of SAMS grouped by layer. Browsers and classroom capture stations talk to the system through the API & Security layer, where every request is authenticated and role-checked before it reaches a business module. The modules store data in MySQL and write to an append-only audit log, and they reach outside systems only through adapters. The module-by-module responsibilities are listed in the table in Section 3.4.

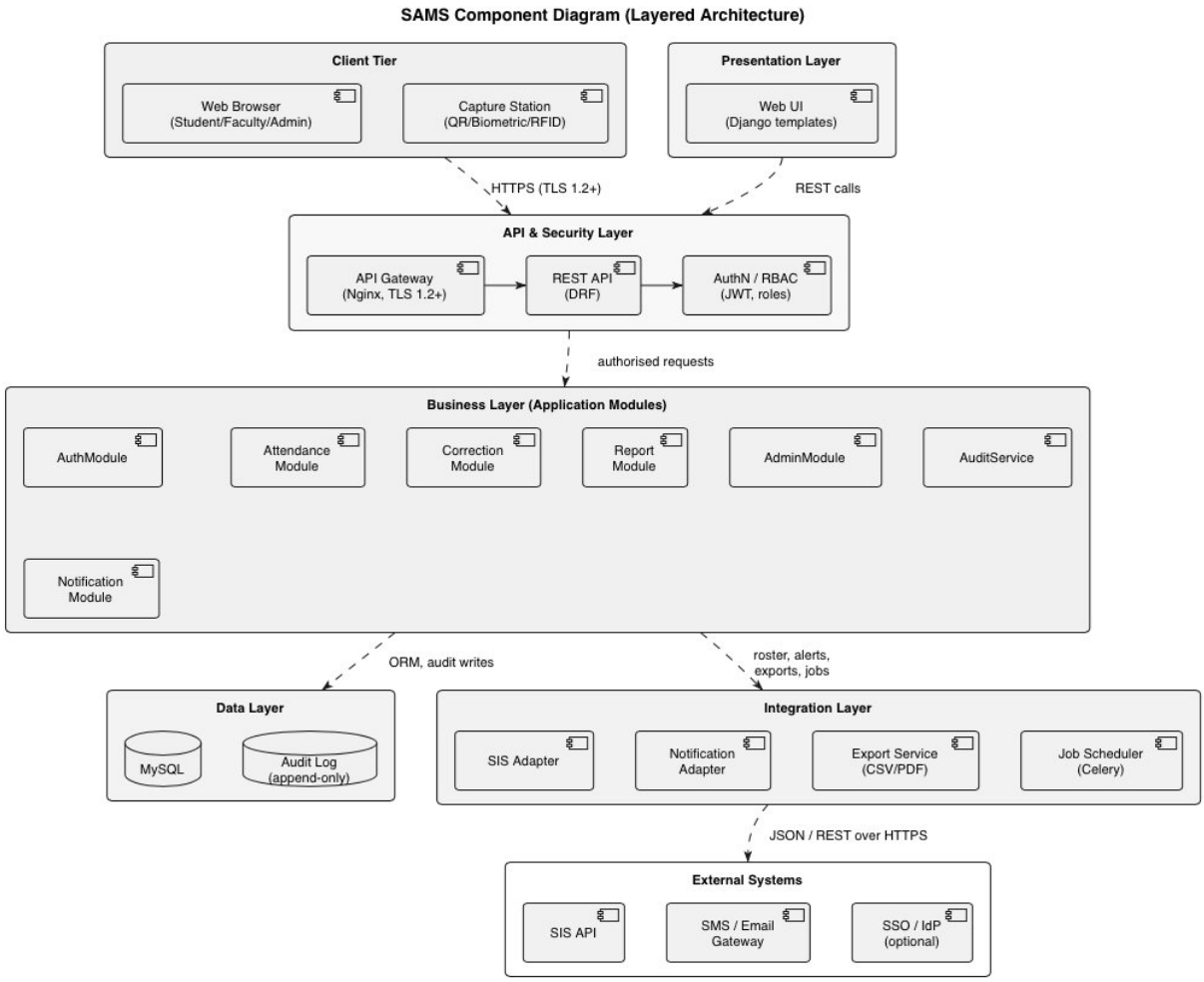


Figure 1 - SAMS component diagram (layered architecture)

Component	Responsibility	Key interfaces	SRS refs
Web UI	Responsive, role-specific dashboards (Student, Faculty, Admin); renders roll-call list, QR display, reports; WCAG 2.1 AA	Calls REST API over HTTPS	SAMS-NF-005, SRS 3.1
Reverse Proxy / API Gateway	Terminates TLS 1.2+, rejects weaker protocols, routes traffic, basic rate limiting	HTTPS in; HTTP to Django (internal)	SAMS-SR-001
REST API (DRF)	Versioned JSON endpoints (/api/v1); request validation; serialisation; error formatting	Serves Web UI and capture stations	SRS 3.4
AuthN / RBAC Middleware	Validates JWT, checks role per endpoint, enforces session idle timeout, independent of UI restrictions	Used by every API view	SAMS-F-002, F-005, SR-005
AuthModule	Login, password hashing (Argon2), lockout after 5 failures, password reset (30-min single-use link/OTP), token issue	Auth API; optional SSO IdP	SAMS-F-001..005, SR-002
AttendanceModule	Opens sessions, manual/QR/biometric capture, duplicate prevention, finalize and lock session	Attendance API; SIS Adapter for roster	SAMS-F-010..015

Component	Responsibility	Key interfaces	SRS refs
CorrectionModule	48-hour faculty correction workflow; admin override with justification; keeps original and corrected values	Correction API; AuditService	SAMS-F-020..023
ReportModule	Attendance % per student/course, defaulters list, daily/weekly/monthly/course-wise reports, analytics dashboard	Reports API; Export Service; Scheduler	SAMS-F-030..033
NotificationModule	Low-attendance and pending-correction alerts to students, guardians and faculty; template and threshold configuration	Notification Adapter	SAMS-F-040..042
AdminModule	Courses, sections, terms, users, roles, attendance policies, leave/absence requests	Admin API; SIS Adapter	SAMS-F-050..053
AuditService	Append-only log of create/edit/override events with user ID, timestamp, reason	Library used by all modules	SAMS-SR-004, NF-004
SIS Adapter	Fetches enrolment/roster/profile from SIS (JSON/HTTPS), caches rosters, retry with backoff	SIS API	SRS 3.3, 3.4
Notification Adapter	Wraps SMS/email provider API; retries and delivery-status logging	SMS/Email Gateway	SRS 3.3
Export Service	Produces CSV and PDF exports matching on-screen totals	Used by ReportModule	SAMS-F-032
Job Scheduler	Celery worker + beat: nightly defaulter job, percentage recalculation, correction-deadline reminders, archival	Message broker (Redis)	SAMS-F-031, F-041
MySQL Database	System of record for SAMS data (users, courses, sessions, attendance, leave, settings); encrypted at rest	Django ORM	SAMS-NF-003, NF-004

3.5 Chosen Architecture Pattern and Rationale

Pattern: layered (n-tier) modular monolith. We split SAMS into Presentation, API & Security, Business, Integration and Data layers. Each Django app is one business module with its own models, services and API views, so the boundaries stay clear even though everything is deployed together. This keeps deployment simple (one Linux server), makes unit, integration and system testing straightforward as planned in the STP, and gives us one central place to enforce security.

Why not microservices? With about 500 concurrent users, at most 80 students per session and a four-person team, the extra effort of running distributed services is not worth it. Attendance marking also needs strong consistency (one record per student per session), which is easiest with a single transactional database. If the load ever grows, the module boundaries make it possible to split services out later.

Architecture Decision Records (summary)

ADR	Decision	Rationale / trade-off
ADR-1	Layered modular monolith instead of microservices	Simplicity, consistency, matches team size and load; less independent scaling
ADR-2	Python Django + Django REST Framework	Mature ORM, built-in auth/admin, security defaults (CSRF, XSS, SQL-injection protection), matches STP test tooling
ADR-3	MySQL relational database	Strong ACID guarantees for attendance records, familiar to the team, listed in SRS and STP environment
ADR-4	JWT access tokens with RBAC enforced at the API layer	Meets SAMS-SR-005 (API enforcement independent of UI) and SAMS-F-005 (20-min idle expiry via short-lived tokens plus refresh)
ADR-5	Append-only audit store; corrections create new versions, never overwrite	Satisfies SAMS-SR-004 and SAMS-NF-004 tamper-evident auditing

ADR	Decision	Rationale / trade-off
ADR-6	Celery + Redis for background jobs	Nightly defaulter computation and notifications must not slow interactive requests
ADR-7	Adapters for SIS and notification gateway with roster caching and retry/backoff	Tolerates brief external outages; keeps SIS as the identity system of record

3.6 Technology Stack & Data Stores

Area	Choice	Notes
Language / framework	Python 3.11+, Django 5, Django REST Framework	Django apps map to modules; DRF for REST endpoints and serializers
Database	MySQL 8 (InnoDB)	Primary store; unique constraint (session_id, student_id) enforces no duplicates; encrypted at rest
Authentication	JWT (SimpleJWT) with Argon2 password hashing (bcrypt as fallback); optional institutional SSO	Passwords never stored in plaintext
Background jobs	Celery with Redis broker	Defaulter job, notifications, archival
Web server	Nginx reverse proxy + Gunicorn on Linux	TLS 1.2+ only; HSTS enabled
Front end	Django templates + responsive CSS/JavaScript; QR generated server-side, scanned via device camera	Accessible, keyboard navigable, high-contrast mode
Exports	Python csv module; ReportLab (or WeasyPrint) for PDF	Exports match on-screen totals
Hardware integration (optional)	QR scanner, fingerprint/RFID readers via classroom capture station	Biometric data stored only as encrypted templates
Testing / tooling	Django test framework, Postman, JMeter, browser dev tools, Jira	As defined in the STP
Data stores	MySQL (operational data), Audit Log Store (append-only table/schema with restricted DB privileges), cached roster tables, archive partition for records older than the active term	5-year retention

3.7 Risks & Mitigations

Risk	Impact	Mitigation
SIS unavailable or slow	Roster unavailable, marking blocked	Local roster cache, retry with exponential backoff, show cached-data banner
Application or network downtime during class	Attendance lost, availability target missed	Health checks, process supervision, DB backups; faculty can finalize late within correction window; monitoring alerts
Peak load at start of class (many QR check-ins)	Response time exceeds 3 s	Indexed queries, connection pooling, stateless app servers behind proxy, load test in JMeter (TC-Perf-01, TC-Scale-01)
Proxy/buddy attendance (QR sharing)	Integrity of records	Short-lived rotating QR token bound to session window, one record per student, optional biometric mode, audit trail
Biometric/privacy breach	Regulatory and trust damage	Store encrypted templates only, field-level encryption, strict RBAC, retention rules
Unauthorised alteration of records	Loss of trust/integrity	Time-boxed corrections, append-only audit log, admin override requires justification
Notification gateway failure	Alerts not delivered	Retry/backoff, delivery-status log, dashboard defaulter list as fallback
Requirement changes during testing	Rework, misaligned tests	Traceability matrix update process; modular design limits change impact

Risk	Impact	Mitigation
Database error or data loss	Loss of records	Daily backups, binary logs, restore drills, transactional writes

3.8 Traceability to Requirements

This table connects each SRS requirement (or group of requirements) to the component that handles it, the place in this SAD where the design is explained (plus the DS-* ID from the SRS traceability matrix where one exists), and the test case that checks it.

Req ID	Requirement (short)	Component(s)	Design ref	Test case(s)
SAMS-F-001	Authenticate user	AuthModule	DS-Auth-01; 4.2.2; 4.3.1	TC-Auth-01
SAMS-F-002	Role-based access control	RBAC Middleware, REST API	3.9; 4.3	TC-Auth-02
SAMS-F-003	Lock after 5 failed logins	AuthModule, AuditService	4.2.2	TC-Auth-03
SAMS-F-004	Password reset (30-min, single use)	AuthModule, Notification Adapter	4.3.1	TC-Auth-04
SAMS-F-005	20-min session timeout	RBAC Middleware	3.9; 4.3.1	TC-Auth-05
SAMS-F-010	Open attendance session	AttendanceModule	DS-Mark-01; 4.2.1	TC-Mark-01
SAMS-F-011	Mark Present/Absent/Late	AttendanceModule	4.3.2	TC-Mark-02
SAMS-F-012	QR self check-in	AttendanceModule	DS-Mark-02; 4.2.1	TC-Mark-03
SAMS-F-013	Biometric / RFID capture	AttendanceModule, Capture Station	4.3.2	TC-Mark-04
SAMS-F-014	Prevent duplicate entries	AttendanceModule, MySQL unique key	4.2.1	TC-Mark-05
SAMS-F-015	Finalize and lock session	AttendanceModule	4.2.1; 4.3.2	TC-Mark-06
SAMS-F-020..023	View/correct within 48 h; admin override; immutable audit	CorrectionModule, AuditService	4.3.2; 3.9	Suite: Record Correction
SAMS-F-030	Attendance % per course	ReportModule	DS-Rep-01; 4.2.3	TC-Rep-01
SAMS-F-031	Flag below-threshold students	ReportModule, Job Scheduler	DS-Rep-02; 4.2.3	TC-Rep-02
SAMS-F-032	Daily/weekly/monthly/course reports, CSV/PDF	ReportModule, Export Service	4.3.3	TC-Rep-03
SAMS-F-033	Analytics dashboard	ReportModule, Web UI	4.3.3; 4.5	TC-Rep-04
SAMS-F-040..042	Low-attendance and deadline notifications; templates	NotificationModule	4.2.3	Suite: Notifications
SAMS-F-050..053	Courses, users, policies, leave requests	AdminModule	3.4; 4.1	Suite: Administration
SAMS-NF-001	3 s response, 90th percentile	API, DB indexing	DS-Perf-01; 3.5	TC-Perf-01
SAMS-NF-002	99.5% availability	Infra, monitoring	3.7; 4.4	Ops monitoring reports
SAMS-NF-003	Encryption at rest/in transit, role-restricted	Gateway, MySQL, RBAC	3.9	TC-Sec-01
SAMS-NF-004	5-year retention, tamper-evident log	AuditService, archive	3.6; 3.9	TC-Ops-01
SAMS-NF-005	WCAG 2.1 AA; 500 concurrent users	Web UI, API	4.5; 3.5	TC-UX-01 / TC-Scale-01
SAMS-SR-001	TLS 1.2+ mandatory	API Gateway	DS-Sec-01; 3.9	TC-Sec-02
SAMS-SR-002	Salted one-way password hash	AuthModule	3.9	TC-Sec-03
SAMS-SR-003	Encrypted biometric templates	AttendanceModule, MySQL	3.9	TC-Sec-04

Req ID	Requirement (short)	Component(s)	Design ref	Test case(s)
SAMS-SR-004	Audit log of create/edit/override	AuditService	3.9; 4.4	TC-Sec-05
SAMS-SR-005	RBAC at API layer (HTTP 403)	RBAC Middleware	3.9; 4.3	TC-Sec-06

3.9 Security Architecture

Security objectives (from SRS 5.1.1): first, keep personal and biometric data away from anyone who should not see or change it. Second, make sure only logged-in users with the right role can view or change attendance, and that every change can be traced back to a person. Third, protect attendance data from accidental loss and tampering so that it stays correct and available.

Threat model (STRIDE)

Threat	Example in SAMS	Mitigation (architecture control)	Req
Spoofing	Stolen credentials; student scanning QR for an absent friend	Argon2-hashed passwords, account lockout after 5 failures, JWT with 20-min idle expiry, rotating session-bound QR token, optional biometric mode	F-001, F-003, F-005, SR-002
Tampering	Editing attendance after the window; modifying API payloads	Session finalize lock, 48-h correction window, server-side validation, append-only audit with original and corrected values, TLS integrity	F-015, F-020..023, SR-001, SR-004
Repudiation	Faculty denies changing a record	Audit entry for every create/edit/override with user ID, timestamp, reason; admin overrides need justification	SR-004, NF-004
Information disclosure	Leak of student or biometric data; verbose errors	TLS 1.2+; encryption at rest; encrypted biometric templates only; generic login errors; no sensitive data in logs; least-privilege DB accounts	NF-003, SR-001, SR-003
Denial of service	Flooding login or QR check-in endpoints at class start	Rate limiting at gateway, lockout, horizontal scaling of stateless app servers, queued background jobs	NF-001, NF-002
Elevation of privilege	Student calls faculty API directly	RBAC enforced in API middleware independent of UI; HTTP 403 for out-of-role calls; admin actions restricted and logged	F-002, SR-005

Security controls summary

- **Transport:** everything travels over TLS 1.2 or higher (web, API, capture stations and server-to-server calls). Older protocols are refused (SAMS-SR-001).
- **Authentication:** passwords are stored as salted Argon2 hashes (SAMS-SR-002). Accounts lock after 5 failed attempts and the event is logged. Password reset links expire and work only once.
- **Authorisation:** every API endpoint checks the user's role and also checks ownership. A student sees only their own records and a faculty member only their own sections (SAMS-SR-005).
- **Data protection:** the database and its backups are encrypted at rest. Biometric data is kept only as encrypted templates, never as raw images (SAMS-SR-003).
- **Auditability:** one AuditService writes to an append-only store, and the database account it uses has restricted privileges (SAMS-SR-004).
- **Secure development:** we validate input on the login, attendance and management forms, use parameterised ORM queries, turn on CSRF protection, scan dependencies, and run the security tests described in section 5.1 of the STP.

4. Design

4.1 Design Overview

SAMS is built as layered modules so that each concern stays separate and easy to maintain. Each business module is a Django app with a service layer that the REST API views call. Shared concerns such as authentication, RBAC, auditing, error handling and logging are written once and reused everywhere. The application tier keeps no state of its own (everything lives in MySQL), so we can run several Gunicorn workers or servers behind Nginx.

Key data entities (logical model)

Entity	Key attributes	Notes
User	user_id, institutional_id, role (STUDENT/FACULTY/ADMIN), password_hash, failed_count, locked_until, email, phone	Identity mirrored from SIS; credentials local or via SSO
Course / Section / Term	course_id, section_id, term_id, faculty_id, schedule slots	Managed by AdminModule
Enrolment	student_id, section_id	Cached from SIS roster
AttendanceSession	session_id, section_id, slot, mode, status (OPEN/FINALIZED), qr_token, opened_at, finalized_at	One session per class meeting
AttendanceRecord	record_id, session_id, student_id, status (PRESENT/ABSENT/LATE), marked_at, source	Unique (session_id, student_id); corrections create new versions
AuditLog	audit_id, actor_id, action, entity, old_value, new_value, reason, timestamp	Append-only
LeaveRequest	leave_id, student_id, dates, reason, status	Approved leave adjusts percentage
Policy / Settings	min_threshold, correction_window_hours, notification templates	Admin-configurable
BiometricTemplate	student_id, encrypted_template	Optional; never raw images
NotificationLog	notification_id, recipient, channel, status, sent_at	Delivery tracking

Design principles

- Each module has one clear job and talks to the others through service interfaces, never by reading another module's tables directly.
- Rules that must always hold are enforced in the database as well as in code, using a unique key for attendance, foreign keys and not-null constraints.
- When something fails, we fail safely and say little: login and permission errors never reveal internal details.
- Calls to scanners and notification services can be safely retried without creating duplicates.

4.2 UML Sequence Diagrams

We modelled the three flows that matter most in SAMS: (1) opening a session, QR check-in and finalizing; (2) logging in, including account lockout; and (3) calculating attendance and sending low-attendance alerts.

4.2.1 Attendance session: open, QR self check-in, finalize

Covers SAMS-F-010, F-012, F-014 and F-015. The faculty member opens a session and the student scans the QR shown in class. The module checks enrolment, the session window and duplicates before saving the record and an audit entry. Finally the faculty member finalizes the session, which locks it.

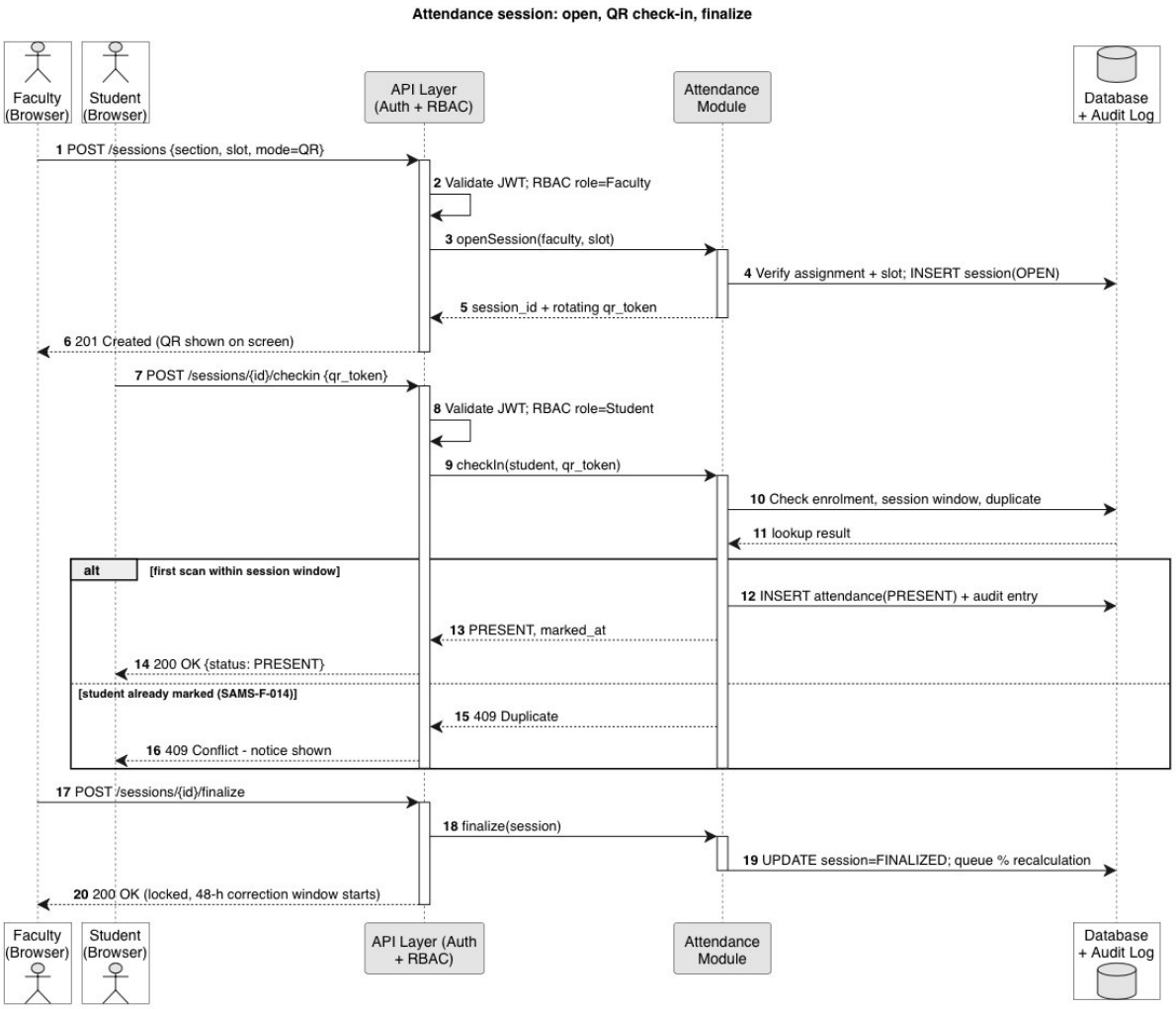


Figure 2 - Sequence diagram: attendance session (open, QR check-in, finalize)

4.2.2 User login with role-based access and account lockout

Covers SAMS-F-001, F-003 and F-005. Valid credentials give the user a JWT that carries their role. After five failed attempts in a row the account is locked and an admin alert is logged.

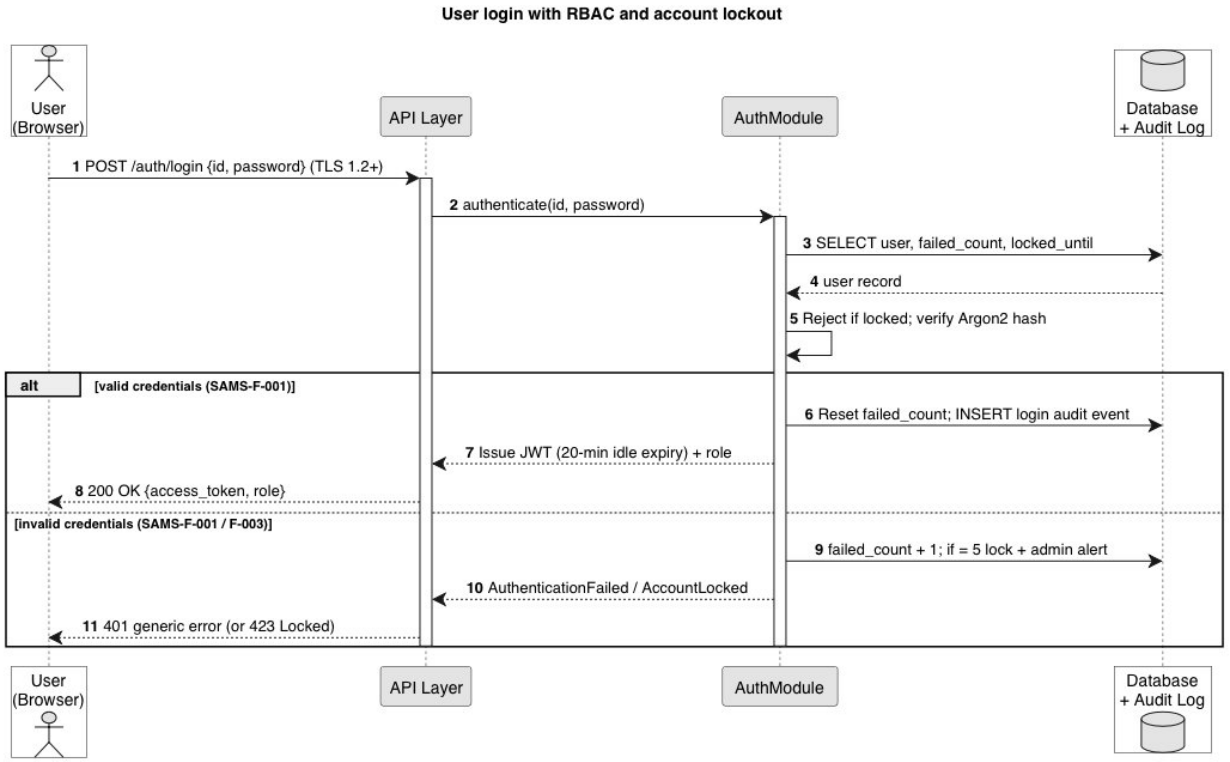


Figure 3 - Sequence diagram: login and lockout

4.2.3 Attendance percentage, defaulter flagging and notification

Covers SAMS-F-030, F-031 and F-040. A scheduled job (and a trigger when a session is finalized) recalculates attendance percentages, updates the defaulters list and sends email/SMS alerts to students and guardians.

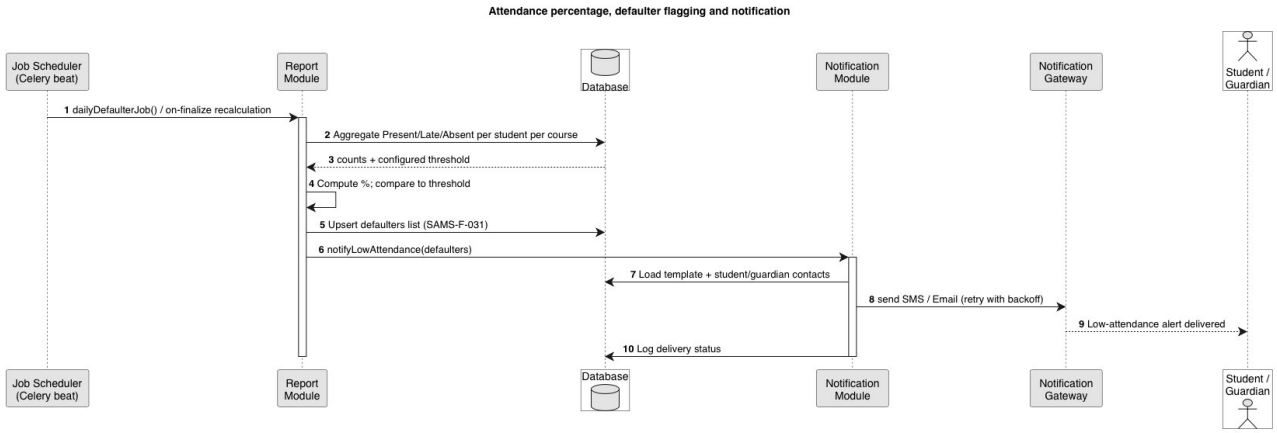


Figure 4 - Sequence diagram: percentage calculation and low-attendance alert

4.3 API Design

Conventions: all endpoints live under /api/v1, use JSON, and work over HTTPS only (TLS 1.2+). Every endpoint except login and password reset needs an Authorization: Bearer <JWT> header. Timestamps are in ISO-8601 UTC, and errors follow the common format in Section 4.4. Below we define the interfaces of three components: AuthModule, AttendanceModule (together with CorrectionModule) and ReportModule.

4.3.1 AuthModule API

Endpoint	Method	Request	Success response	Errors
/auth/login	POST	{institutional_id, password}	200 {access_token, expires_in, role}	401 invalid credentials (generic); 423 account locked; 429 too many requests
/auth/logout	POST	(token in header)	204 No Content	401 invalid/expired token
/auth/password-reset/request	POST	{email}	202 Accepted (always, to avoid account enumeration)	429 too many requests
/auth/password-reset/confirm	POST	{token, new_password}	200 {status: "password_updated"}	400 weak password; 410 link expired or already used (30-min, single use)

Example - POST /api/v1/auth/login

```
Request : {"institutional_id": "PES2UG24CS001", "password": "********"}
Response: 200 {"access_token": "<jwt>", "expires_in": 1200, "role": "STUDENT"}
Error   : 401 {"error": {"code": "AUTH_INVALID", "message": "Invalid credentials"}}
```

4.3.2 AttendanceModule and CorrectionModule API

Endpoint	Method	Role	Request	Success response	Errors
/sessions	POST	Faculty	{section_id, slot_id, mode: MANUAL|QR|BIOMETRIC}	201 {session_id, status: OPEN, qr_token?}	403 not assigned faculty; 409 session already open; 422 outside scheduled slot
/sessions/{id}/records	PUT	Faculty	{records: [{student_id, status: PRESENT|ABSENT|LATE}]}	200 {updated: n}	403; 404 session; 409 session finalized
/sessions/{id}/checkin	POST	Student	{qr_token}	200 {status: PRESENT, marked_at}	403 not enrolled; 409 duplicate; 410 window closed; 422 invalid token
/sessions/{id}/capture	POST	Capture station	{device_id, reading}	200 {student_id, status: PRESENT}	404 no matching template/card; 409 duplicate
/sessions/{id}/finalize	POST	Faculty	(none)	200 {status: FINALIZED, correction_deadline}	403; 409 already finalized
/records/{id}	PATCH	Faculty / Admin	{status, reason}	200 {record_id, old_status, new_status}	403 window expired (faculty); 400 reason required; 404
/records/{id}/override	POST	Admin	{status, justification}	200 {record_id, audit_id}	403; 400 justification required

Example - student QR check-in:

```
POST /api/v1/sessions/812/checkin
Request : {"qr_token": "a93f..."}
Response: 200 {"status": "PRESENT", "marked_at": "2026-10-09T04:15:22Z"}
Error   : 409 {"error": {"code": "DUPLICATE_ATTENDANCE", "message": "Attendance already recorded"}}
```

4.3.3 ReportModule API

Endpoint	Method	Role	Request / parameters	Success response	Errors
/students/{id}/attendance	GET	Student	?course_id&from&to	200 {course_id,	403 not permitted;

Endpoint	Method	Role	Request / parameters	Success response	Errors
		(own), Faculty, Admin		present, total, percentage}	404
/courses/{id}/defaulters	GET	Faculty, Admin	?threshold (optional)	200 {threshold, students: [{student_id, percentage}]}	403; 404
/reports	GET	Faculty, Admin	?type=daily|weekly|monthly|course&from&to&format=json|csv|pdf	200 report body or file download	400 bad parameters; 403
/analytics/trends	GET	Admin	?course_id&section_id&from&to	200 {series: [{date, percentage}]}	403; 400

4.4 Error Handling, Logging & Monitoring

Error handling

Every error comes back in the same JSON format: {"error": {"code": "...", "message": "...", "details": [...]}}. The messages are standard and safe for users to see, and we never send stack traces or internal IDs.

HTTP status	Code	Meaning
400	VALIDATION_ERROR	Missing or malformed input (e.g. missing reason for correction)
401	AUTH_INVALID / TOKEN_EXPIRED	Invalid credentials (generic) or expired/absent token
403	FORBIDDEN	Role not permitted or object not owned (SAMS-SR-005)
404	NOT_FOUND	Resource does not exist or not visible to the caller
409	DUPLICATE_ATTENDANCE / STATE_CONFLICT	Duplicate record, session already open or finalized
410	WINDOW_CLOSED / LINK_EXPIRED	Check-in window closed or reset link expired
422	UNPROCESSABLE	Outside scheduled slot or invalid QR token
423	ACCOUNT_LOCKED	Account locked after 5 failed attempts
429	RATE_LIMITED	Too many requests
503	DEPENDENCY_UNAVAILABLE	SIS or gateway unavailable; cached data used where possible

Logging

- **Application log** (structured JSON): request ID, user ID, endpoint, status and response time. We never log passwords, tokens, QR tokens, biometric data or full personal details.
- **Audit log** (append-only): every attendance creation, edit and override with who did it, when, the old and new value, and the reason. Logins, failed logins and lockouts are recorded too (SAMS-SR-004).
- **Integration log**: every call to the SIS and the notification gateway, with the outcome and the number of retries.
- Application logs are kept as long as the institution's policy says, and audit records are kept for at least 5 years (SAMS-NF-004).

Monitoring

Metric	Target / alert	Source
Application uptime / availability	≥ 99.5% monthly; alert on health-check failure	Health endpoint, uptime monitor
Attendance submission latency (p90)	≤ 3 s; alert if exceeded for 5 min	API metrics
Failed login and lockout rate	Alert on spike (possible attack)	Audit log
Notification delivery failure rate	Alert above threshold; retry queue depth	NotificationLog, Celery
SIS sync / cache age	Alert if roster cache stale beyond policy	SIS Adapter
Background job success (nightly defaulter job)	Alert on failure or missed run	Celery beat/worker
Database health and backup status	Alert on replication/backup failure, disk usage	DB monitoring

4.5 UX Design

- **Dashboards for each role:** students see their attendance percentage per course, their history, a leave request form and a low-attendance warning. Faculty can open a session, use the roll-call list with Present/Absent/Late toggles, show the QR code, see corrections with a deadline countdown, and view class analytics. Admins manage courses, users, policies, institution-wide reports and overrides.
- **Quick marking:** the roll-call list starts with a default status, there is a bulk action (mark all present) and a single Submit button. We aim for confirmation in under 3 seconds.
- **QR check-in:** students open the page on their phone and scan the QR code shown on the classroom screen. They get clear messages for success, duplicate scans and a closed window.
- **Accessibility (WCAG 2.1 AA):** screen-reader labels, full keyboard navigation, a high-contrast mode, readable font sizes and colour contrast, and no status shown by colour alone (icons and text go with the colours).
- **Responsive layout:** mobile-first for students, and tuned for desktop and tablet for faculty and capture stations.
- **Clear feedback:** the system asks for confirmation before a session is finalized, shows a lock icon afterwards, asks for a reason when a record is corrected or overridden, and warns the user before the 20-minute idle timeout logs them out.

4.6 Open Issues & Next Steps

#	Issue / next step	Owner / status
1	The SRS and the Test Plan do not match in a few places. The STP uses "SAM-" IDs and test IDs like TC-AUTH-01, while the SRS uses "SAMS-" and TC-Auth-01. The STP also targets 95% of requests within 3 s, 99% availability and 100 concurrent users, but the SRS says 90% within 3 s, 99.5% and 500 users. This SAD follows the SRS, and the team should align the two documents.	Team Lead + QA Lead - open
2	The SRS lists requirement ranges for sections 4.3, 4.5 and 4.6 (F-020..023, F-040..042, F-050..053) without individual tables. We should list each one and add test cases and DS IDs to the RTM.	Team - open
3	The STP leaves hardware devices (QR/biometric) out of testing, but the SRS includes them as optional. We need to confirm what the MVP includes and decide how to test with mock devices.	Team - open
4	Decide between SSO and local login for deployment, and decide how often the QR token rotates and how long the check-in window stays open.	Team / Coordinator - open
5	Decide where guardian contact details come from (SIS or manual entry), how consent is handled, and how long biometric data is kept.	Admin / Institution - open
6	Ideas for later: face recognition or location-based check-in, a native mobile app, timetable/LMS integration, and predictive analytics to spot students at risk.	Backlog
7	Next steps: finalise the database schema and migrations, build the Auth and Attendance modules first, set up CI with unit tests, run JMeter load tests against the NFR-001 and NF-005 targets, and	Team - planned

#	Issue / next step	Owner / status
	finish the security tests from STP section 5.1.	

5. Appendices

5.1 Glossary

Term	Definition
Attendance session	A single class meeting for which attendance is recorded; OPEN until finalized
Finalize	Faculty action that locks session attendance pending the correction window
Correction window	Period (default 48 hours) in which faculty may correct finalized attendance
Defaulter	Student whose attendance percentage is below the institution-configured threshold
Override	Administrator change to a record outside the correction window, with justification
Audit log	Append-only record of who changed what, when and why
Capture station	Classroom tablet/kiosk connected to QR, biometric or RFID devices
SAMS, SIS, RBAC, TLS, JWT, ADR, STRIDE, WCAG, RTM	See Section 1.4

5.2 References

- Team 1, *Software Requirements Specification - Student Attendance Management System*, v1.0, 13-09-2026.
- Team 1, *Software Test Plan - Student Attendance Management System*, v1.0, 16-09-2026.
- ISO/IEC/IEEE 42010:2022, Systems and software engineering - Architecture description.
- OWASP Application Security Verification Standard (ASVS) and OWASP Top 10.
- NIST SP 800-160, Systems Security Engineering.
- W3C, Web Content Accessibility Guidelines (WCAG) 2.1.
- Django and Django REST Framework official documentation.

5.3 Tools

We drew the diagrams in draw.io, using PlantUML for the UML sequence and component diagrams. API documentation and testing use Swagger/OpenAPI and Postman, load testing uses JMeter, defects are tracked in Jira, and the code lives in Git.